

Introduction to recommender systems

Prof. Luca Cagliero
Dipartimento di Automatica e Informatica
Politecnico di Torino



**Politecnico
di Torino**

Lecture goal

- Recommender systems fundamentals
- Content-based recommendation
- Collaborative filtering
- Hybrid methods

Objective

Recommender systems try to identify the need and preferences of users, filter the huge collection of data accordingly and present the best suited option before the users by using some well-defined mechanism

Objective

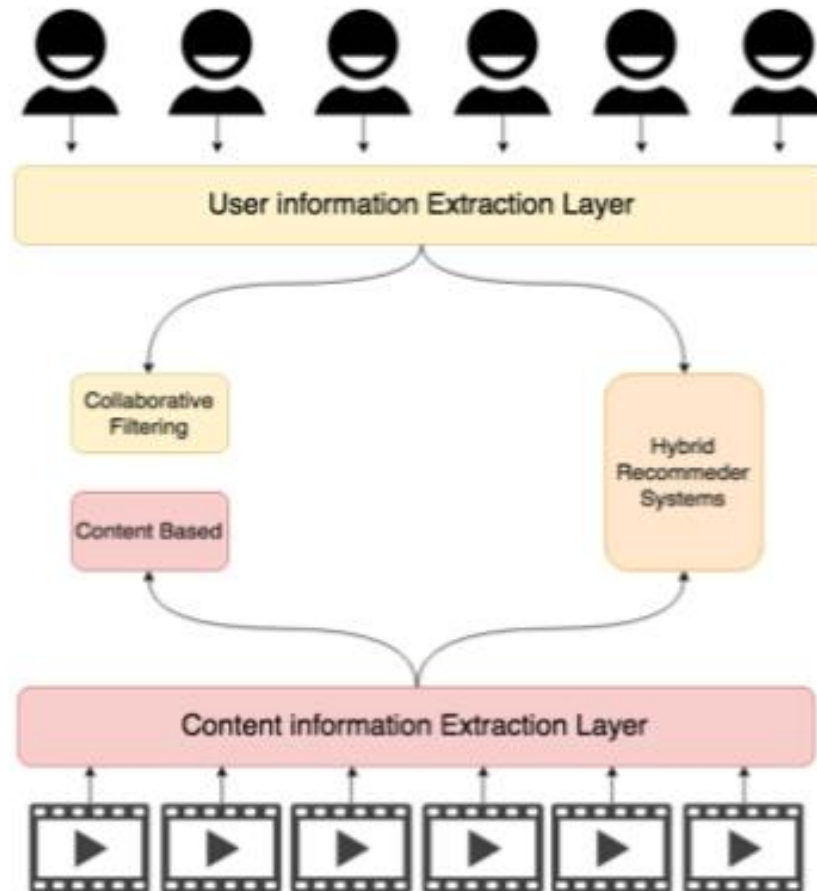
- Predict user responses or preferences
- Examples of application contexts
 - E-commerce
 - recommend the item to buy
 - On demand services
 - recommend the movie to watch
 - Content curation
 - recommend the news articles to read
 - Tourism and hospitality
 - recommend the places to visit or the hotels to book



System categorization

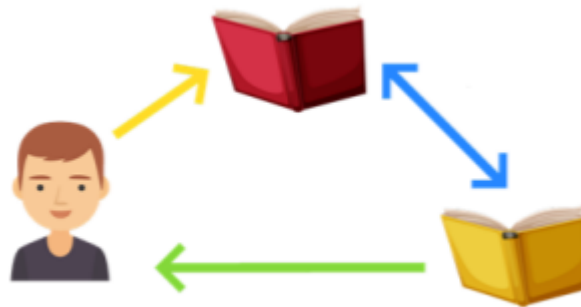
- Content-based systems
- Collaborative filtering
- Hybrid approaches

Framework



Content-based recommender systems

- Recommendations are based on the characteristics of the items recommended
- Recommend an item to a user similar to those that she/he previously rated highly
 - E.g., if a user frequently watches horror movies then it recommends a movie within the same category



Content-based recommender systems: key idea



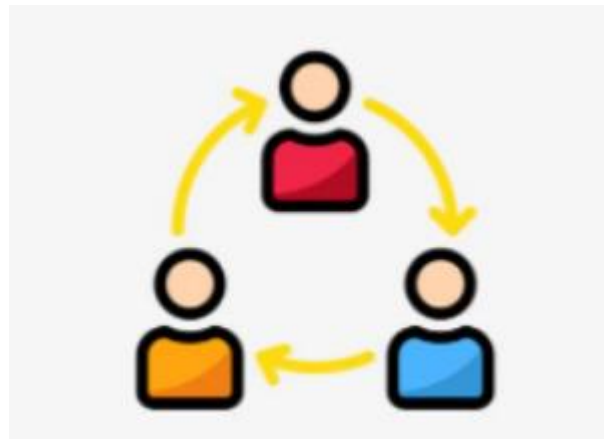
Content types

- Products
- Locations
- Images
- Tags
- Videos
- Teaching materials
- Etc.



Collaborative filtering

- Recommendations rely on user similarities
- If two users are similar to each other then they tend to prefer similar items
- Recommend to a user the items that are most popular for similar users
 - E.g., if football players like Messi videos then it likely recommends a Messi videos to a user of category football player



Hybrid systems

- Combine content-based with collaborative filtering approaches
- Possible solutions:
 - **Ensemble methods:** Build separate methods and then combine the recommendations
 - **Nested methods:** Integrate either content-based characteristics into a collaborative approach or collaborative characteristics into a content-based approach
 - **Mixed strategies:** Build a general model that integrates both content-based and collaborative characteristics



Utility matrix

- Rows: users
 - A, B, C, D
- Columns: items
 - HP1, HP2, and HP3 for Harry Potter I, II, and III, TW for Twilight, and SW1, SW2, and SW3 for Star Wars episodes 1, 2, and 3
- Values: ratings
 - from 1 (very low) to 5 (very high)

Utility matrix

	HP1	HP2	HP3	TW	SW1	SW2	SW3
<i>A</i>	4			5	1		
<i>B</i>	5	5	4				
<i>C</i>				2	4	5	
<i>D</i>		3					3

Example taken from 'Mining of Massive Datasets' by Jure Leskovec, Anand Rajaraman, and Jeff Ullman. <http://www.mmds.org/>

Utility matrix

	Item 1	Item 2	Item 3	Item 4
User 1	0	4	5	5
User 2	1	5	5	-
User 3	5	2	1	1
User 4	4	2	1	-

Scores: 0 - 5

Utility matrix

	Item 1	Item 2	Item 3	Item 4
User 1	0	4	5	5
User 2	1	5	5	4
User 3	5	2	1	1
User 4	4	2	1	1

Scores: 0 - 5

Feedback types

- Feedback can be classified as
 - Explicit: explicitly provided by the users
 - Implicit: deducted from users' behaviors

Utility matrix population

- Collecting utility matrix values is necessary
- How to get item rates?
 1. User annotation: ask users to rate items
 - Complete
 - Partial (some users are unwilling to provide ratings)
 2. Rate inference (implicit feedback)
 - deduce item ratings from user actions (e.g., time to leave, click rate, Auditel scores)
 - Infer item rankings using Machine Learning models (independently of the recommender system)

Problem statement

- U : set of users
- I : set of items
- R : ordered set of ratings
- $F(\cdot): U \times I \rightarrow R$

Problem statement

- Given U , I , R , and the known known values for $F(\cdot)$
- Predict the unknown values for $F(\cdot)$
- For each user $u \in U$ the recommender system returns the list of top- K items with highest rate
 - K is an input parameter

Problem statement

- The purpose is typically to predict the cases when rating $r \in R$ is very high or very low, e.g.,
 - Predict the items that a user really likes is relevant to plan marketing strategies
 - Predict the movie categories that a user dislikes help preventing service drop out

Main challenges

- Deal with sparse utility matrices
- Cold start
- First rater
- Popularity bias

Sparsity

- Many user and item combinations are missing
- Hard to build unified item and user profiles

First rater

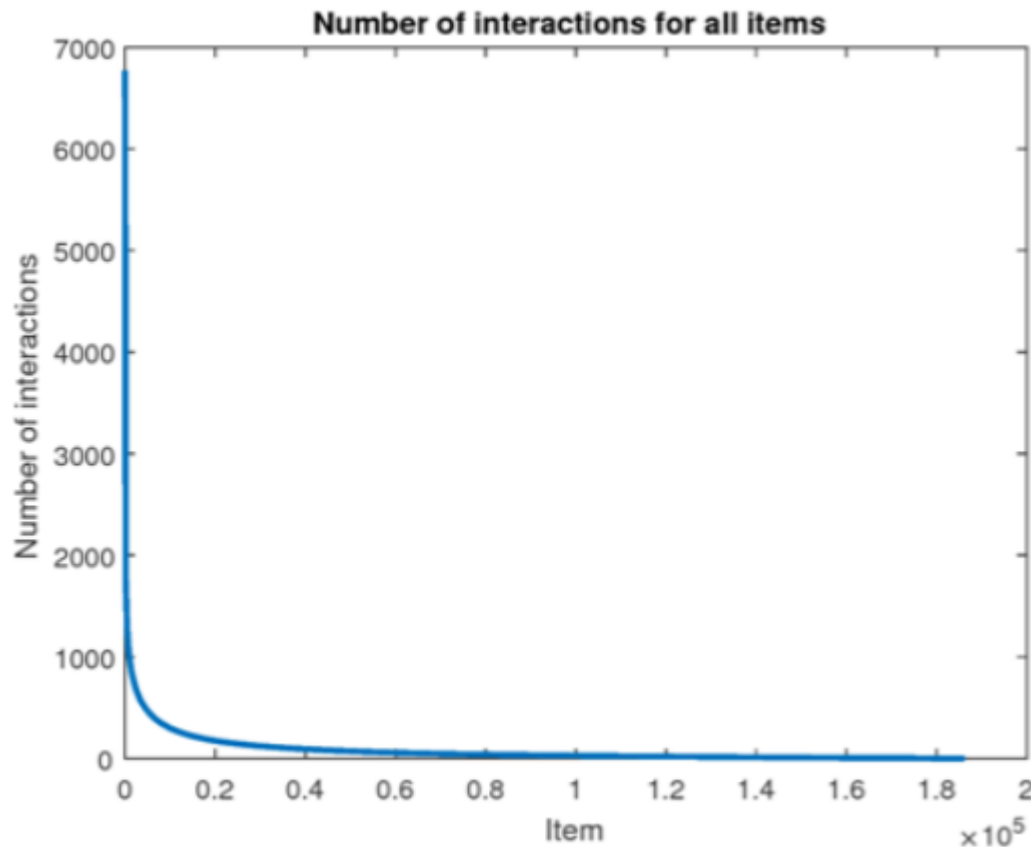
- Recommend items with unknown ratings
- Items that have never been rated cannot be recommended
- Need for ad hoc solutions
 - Typically content-based

Cold start problem

- Recommend items to new users
 - User ratings $F(\cdot)$ are not available
 - User preferences are unknown

The long-tail phenomenon

- Relevant to both cold start and first rater problems



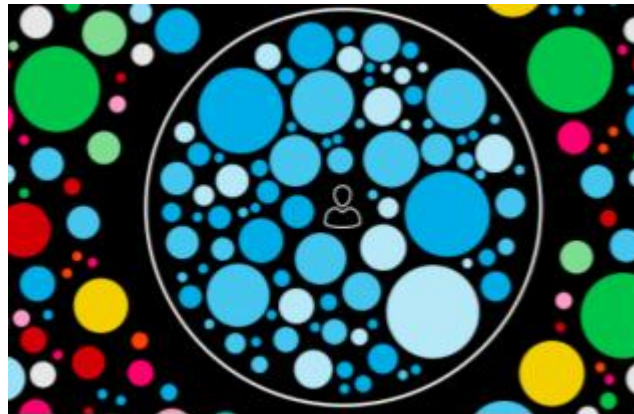
Source: [https://en.wikipedia.org/wiki/Cold_start_\(recommender_systems\)](https://en.wikipedia.org/wiki/Cold_start_(recommender_systems))

The long-tail phenomenon

- Item popularity is rather variable
- It is not possible to tailor the offer to each single customer
- This motivates the need for effective recommender systems

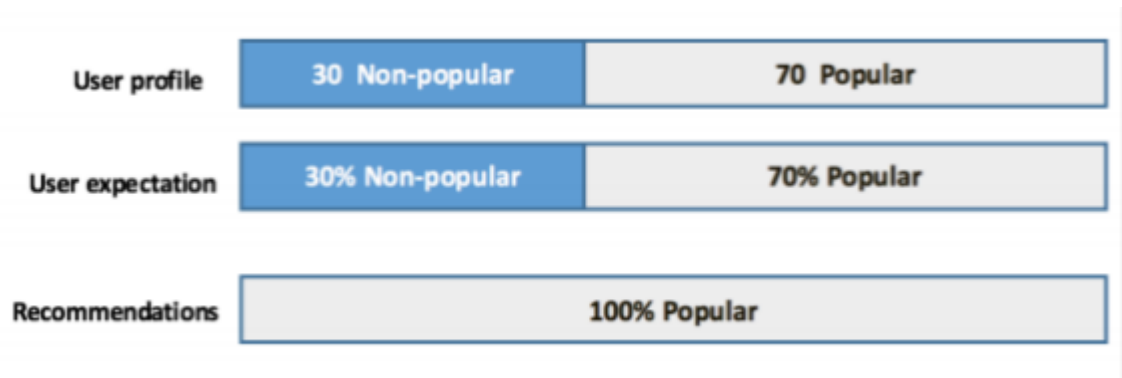
Filter bubble

- Content-based filtering could be biased
 - It recommends only the items that are strictly related to the firstly evaluated items



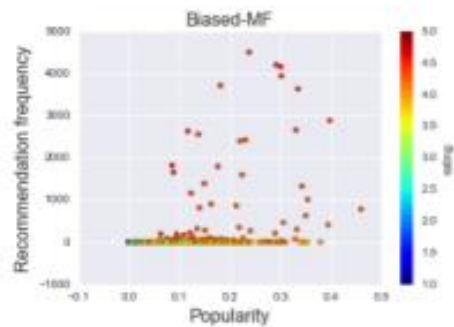
Popularity bias

- Frequently rated items get over-exposure
- Rarely rated items get under-exposure



Popularity bias

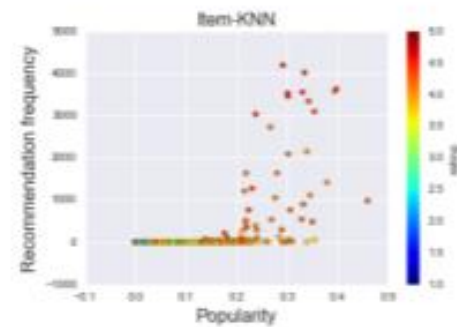
- Frequently rated items get over-exposure
- Rarely rated items get under-exposure



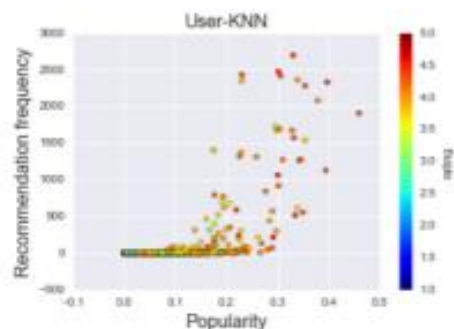
(a) Biased MF



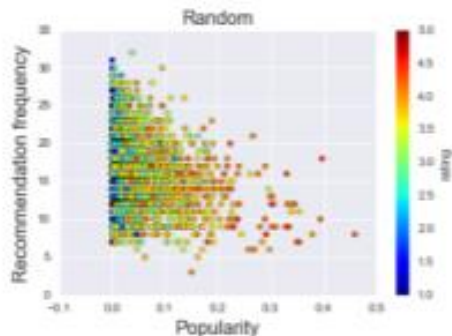
(b) SVD++



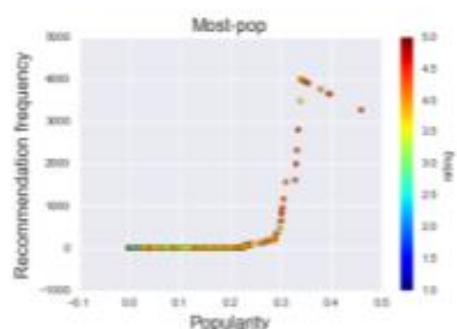
(c) Item KNN



(d) User KNN



(e) Random



(f) Pop

The Unfairness of Popularity Bias in Music Recommendation: A Reproducibility Study. Dominik Kowald, Markus Schedl, Elisabeth Lex. ECJR 2019

Description of the main system categories

- **Content-based systems**
- Collaborative filtering
- Hybrid approaches

Content-based recommender systems

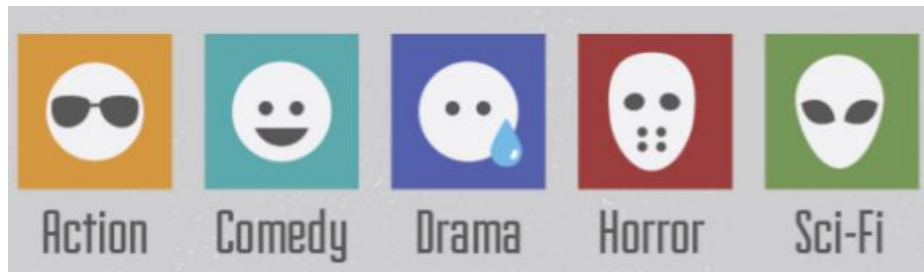
- Item profile $\mathbf{V}_i$: vector representation of an item $i \in I$
- Approach
 - Define a set of item features
 - Compute the item profile of each candidate item $\mathbf{V}_{ci}$
 - Given a user $u \in U$ pick the top-N highly rated items $i^u_{1st}, i^u_{2nd}, \dots, i^u_{n-th}$
 - Compute the item profiles of the highly rated items
 - Compute pairwise similarities with all j-th candidate items

$$\text{sim}(\mathbf{V}_{i_k^u}, \mathbf{V}_{j_k^u})$$

- Recommend the most similar candidates

Content-based recommender systems

- Example: movie recommendation
- Possible item features
 - Genre
 - Production year
 - Main character name
 - Director name
 - Parental control on/of



Content-based recommender systems

- Example: document recommendation
- Item profile: tf-idf scores for each word in the documents
- $n_{i,j}$: number of occurrences of word i in document j
- d_j : number of words in document j
- $|D|$: number of documents in the collection

$$tf - idf_{i,j} = tf_{i,j} \cdot idf_i$$

$$tf_{i,j} = \frac{n_{i,j}}{d_j}$$

$$idf_i = \log_{10} \frac{|D|}{|\{d:i \in d\}|}$$

Vector similarities

- Euclidean distance $||\mathbf{A} - \mathbf{B}||^2 = (\mathbf{A} - \mathbf{B})^T (\mathbf{A} - \mathbf{B})$

- Cosine similarity $\frac{\mathbf{A} \cdot \mathbf{B}}{||\mathbf{A}|| ||\mathbf{B}||} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \sqrt{\sum_{i=1}^n b_i^2}}$

$$Eucl(\mathbf{A}, \mathbf{B}) = ||\mathbf{A} - \mathbf{B}||^2 = (\mathbf{A} - \mathbf{B})^T (\mathbf{A} - \mathbf{B}) = 2 (1 - \cos(\mathbf{A}, \mathbf{B}))$$

Jaccard similarity and dissimilarity

- a: number of dimensions that equal 1 for both vectors V_i and V_j
- b: number of dimensions that equal 0 for vector V_i but equal 1 for vector V_j
- c: number of dimensions that equal 1 for vector V_i but equal 0 for vector V_j
- d: number of dimensions that equal 0 for both vectors V_i and V_j

$$JS_{i,j} = \frac{a}{a+b+c}$$

$$JD(V_i, V_j) = \frac{b+c}{a+b+c} = 1 - JS_{i,j}$$

Content-based recommendation

- Item similarity can be computed at the embedding level
 - Word embeddings (e.g., FastText), sentence-level embedding, or contextualized embeddings (e.g., BERT)
 - You can compare multilingual documents
 - You can compare multimodal items
 - E.g., Visually-rich documents, videos, images
- Instead of comparing single items, you can compare item groups or sequences of items

Description of the main system categories

- Content-based systems
- **Collaborative filtering**
- Hybrid approaches

Collaborative filtering – key steps

1. **Utility matrix population:** Collect the ratings for both target and annotating users
2. **Neighbor formation:** Compute the similarity between target and annotating users
3. **Selection and ranking of candidate items:** Pick the items that are highly rated by similar users
4. **Rate prediction:** For each candidate item forecast the rate that would be given by the target user
5. **Item ranking:** Rank candidate items and return the top-N recommended items

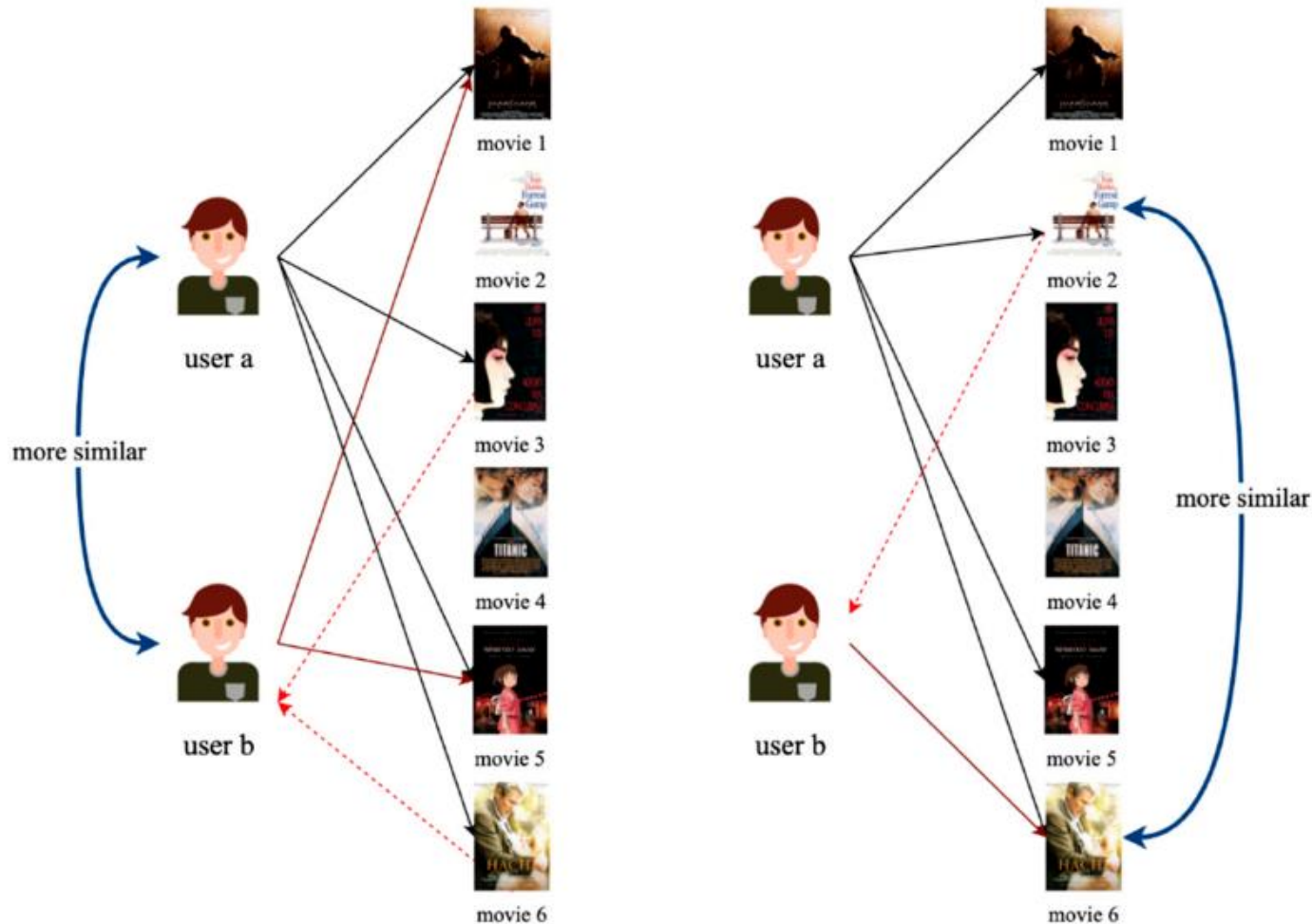
Collaborative filtering – main approaches

1. **Memory-based approaches:** Exploit the ratings in the utility matrix to compute similarities between users/items
2. **Model-based approaches:** Exploit a rating estimate or learn a rating estimator and then apply it to forecast the user rating

Collaborative filtering – main approaches

1. **User-based neighbor models:** assign to the target item a rate similar to those assigned by the most similar users
2. **Item-based neighbor models:** assign to the target item a rate similar to those assigned by other users to the most similar items

User- vs. item-based collaborative filtering



User-based collaborative filtering: problem statement

- s_{ux} : similarity of users u and x
- r_x : vector of the user x ratings
- $N(u, i)$: set of K user that are most similar to u who rated item i

$$\mathcal{F}(u, i) = \frac{1}{K} \sum_{x \in N} \mathcal{F}(x, i) = \frac{\sum_{x \in N(u, i)} s_{ux} \cdot \mathcal{F}(x, i)}{\sum_{x \in N(u, i)} s_{ux}}$$

Item-based collaborative filtering: problem statement

- Find the items that are most similar to the target item
- Estimate the rating for the target item based on the ratings for similar items

Item-based collaborative filtering

- s_{ux} : similarity of users u and x
- $F(u,i)$: rating of user u on item i
- $N(u, i)$: set of items rated by user u and similar to item i

$$\mathcal{F}(x, i) = \frac{\sum_{j \in N(x, i)} s_{ij} \cdot \mathcal{F}(x, j)}{\sum_{j \in N(x, i)} s_{ij}}$$

Model-based collaborative filtering

- Classical approaches
 - Dimensionality reduction
 - Probabilistic models
 - Machine Learning models
 - Graph mining
 - Clustering
 - Association rule mining
 - RNNs
 - Large Language Models

Dimensionality reduction

- Goal
 - Estimate blank entries in the utility matrix
- Approach
 - Singular Value Decomposition (SVD)
- Idea
 - Reduce the utility matrix to capture the most important aspects of the data, ignore the rest
 - Estimate ratings based on the reduced data model

Singular Value Decomposition

Decomposition of matrix $\mathbf{A} \in \mathbb{R}^{N \times M}$ of rank $r \leq \min(N, M)$

$$\mathbf{U} = \left[\underbrace{u_1, u_2, \dots, u_r}_{\mathbf{U}_r} \mid \underbrace{u_{r+1}, \dots, u_N}_{\mathbf{U}_r} \right] = [\mathbf{U}_r \mid \mathbf{U}_r]$$

$$\mathbf{V} = \left[\underbrace{v_1, v_2, \dots, v_r}_{\mathbf{V}_r} \mid \underbrace{v_{r+1}, \dots, v_M}_{\mathbf{V}_r} \right] = [\mathbf{V}_r \mid \mathbf{V}_r]$$

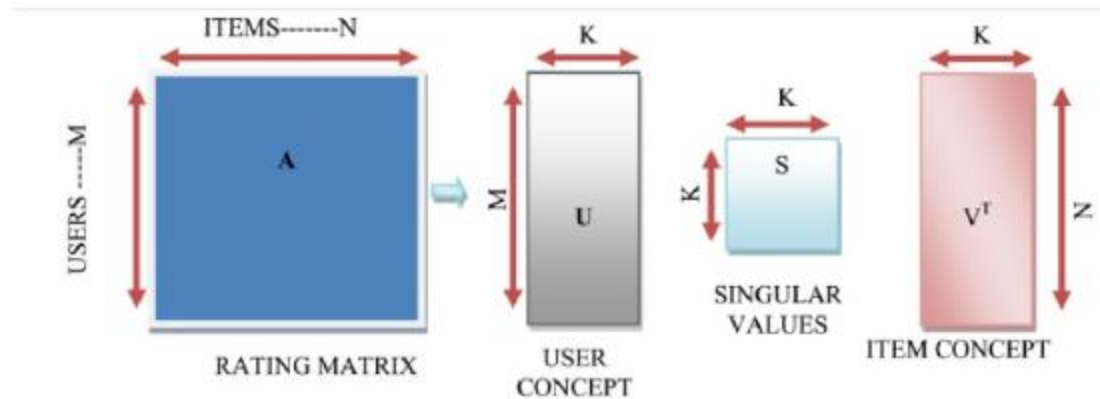
$$\mathbf{A} = [\mathbf{U}_r \mid \mathbf{U}_r] \begin{bmatrix} \boldsymbol{\Sigma}_r & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{V}_r^T \\ \mathbf{V}_r^T \end{bmatrix} = \mathbf{U}_r \boldsymbol{\Sigma}_r \mathbf{V}_r^T$$

$$\begin{aligned} \mathbf{A} &= \sum_{i=1}^r \sigma_i u_i u_i^T = u_1 v_1^T \sigma_1 + u_2 v_2^T \sigma_2 + u_3 v_3^T \sigma_3 + u_4 v_4^T \sigma_4 \\ &\quad + \dots + u_r v_r^T \sigma_r, \quad \text{rank}(\mathbf{A}) = r \end{aligned}$$

$$\begin{aligned} \mathbf{B} &= \sum_{i=1}^k \sigma_i u_i u_i^T = u_1 v_1^T \sigma_1 + u_2 v_2^T \sigma_2 + u_3 v_3^T \sigma_3 + u_4 v_4^T \sigma_4 \\ &\quad + \dots + u_k v_k^T \sigma_k \quad \text{rank}(\mathbf{B}) = k \end{aligned}$$

$$\mathbf{A} - \mathbf{B} = \mathbf{U} \boldsymbol{\Sigma}_A \mathbf{V}^T - \mathbf{U} \boldsymbol{\Sigma}_B \mathbf{V}^T = \mathbf{U}^T [\boldsymbol{\Sigma}_A - \boldsymbol{\Sigma}_B] \mathbf{V}^T \rightarrow \|\mathbf{A} - \mathbf{B}\| = \sigma_{k+1}$$

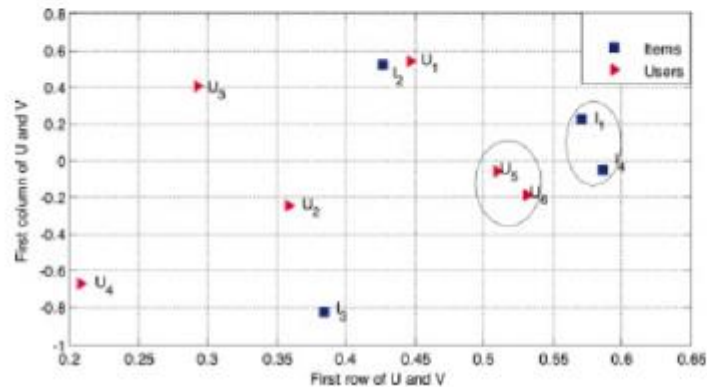
Singular Value Decomposition



A recommender system based on collaborative filtering using ontology and dimensionality reduction techniques. Mehrbakhsh Nilashi, Othman Ibrahim, Karamollah Bagherifard. Expert Systems with Applications. 2018

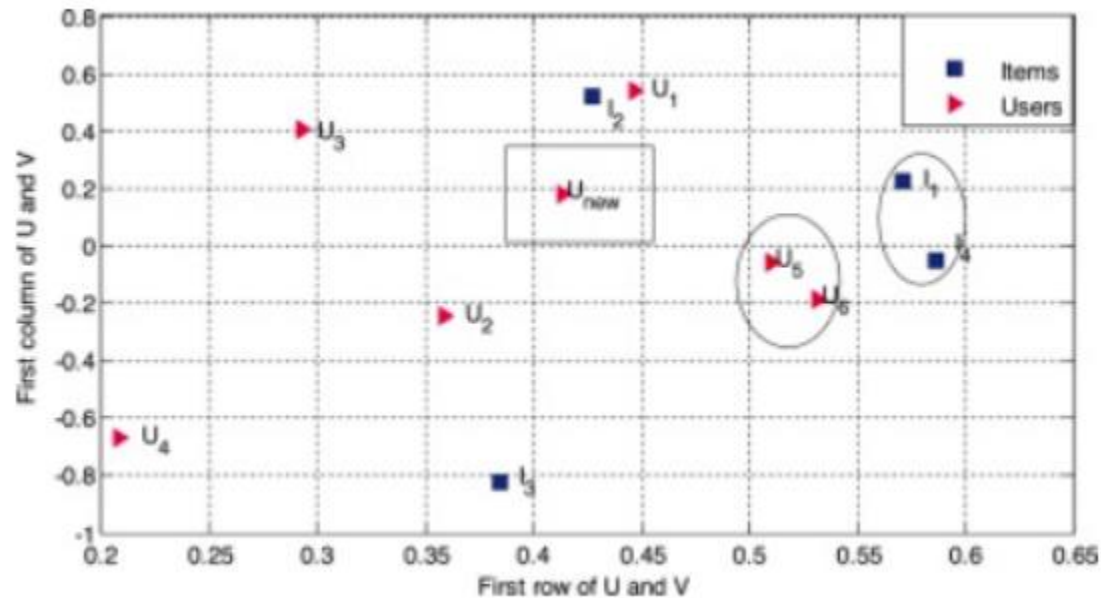
Singular Value Decomposition

	Item 1	Item 2	Item 3	Item 4
User 1	5	5	0	5
User 2	5	0	3	4
User 3	3	4	0	3
User 4	0	0	5	3
User 5	5	4	4	5
User 6	5	4	5	5



A recommender system based on collaborative filtering using ontology and dimensionality reduction techniques. Mehrbakhsh Nilashi, Othman Ibrahim, Karamollah Bagherifard. Expert Systems with Applications. 2018

Singular Value Decomposition



A recommender system based on collaborative filtering using ontology and dimensionality reduction techniques. Mehrbakhsh Nilashi, Othman Ibrahim, Karamollah Bagherifard. Expert Systems with Applications. 2018

SVD for recommendation

- Users 5 and 6 and items 1 and 4 are located very close to each other
- SVD for dimensionality reduction can be applied effectively to reveal the users that have similar taste and form neighbors for items and users
- SVD approximates the missing ratings based on the following matrix factorization

$$\hat{r} = \left(\mathbf{U}_k \mathbf{S}_k^{\frac{1}{2}} \right)_u \cdot \left(\mathbf{S}_k^{\frac{1}{2}} \mathbf{V}_k' \right)_i$$

Probabilistic approaches: example

- Compute the probability that a user $u \in U$ likes item $i \in I$
- Recommend the item with maximal likelihood

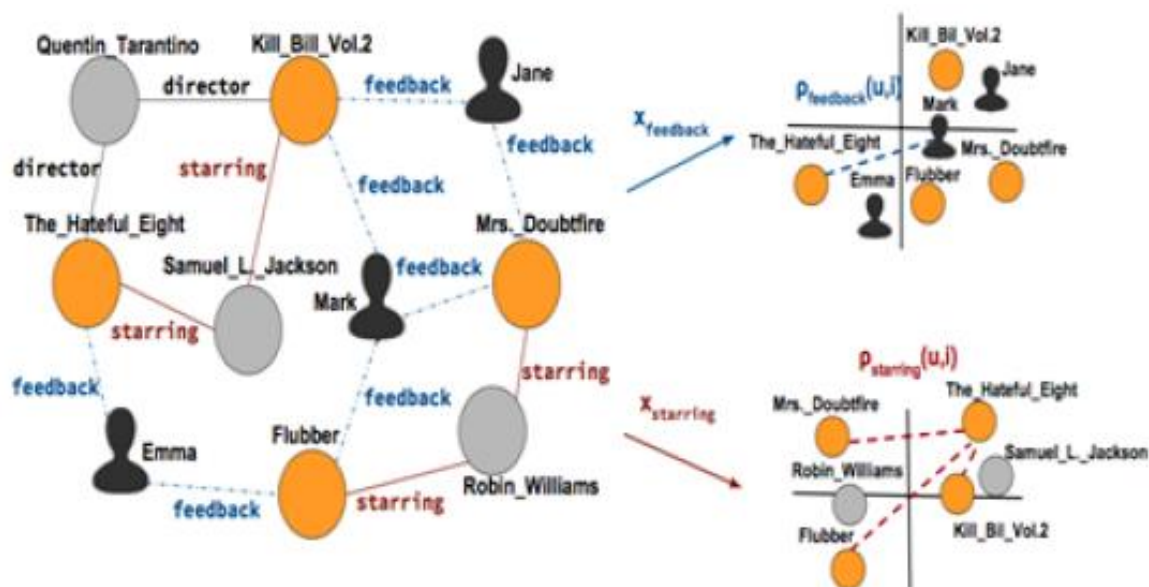
$$\arg_{i \in I} \max P(\mathcal{F}(i) = HR | u)$$

- Bayes theorem $P(\mathcal{F}(i) = HR | u) = \frac{P(u | \mathcal{F}(i) = HR) P(\mathcal{F}(i) = HR)}{P(u)}$
where HR is the highest rating
- $P(u)$ is irrelevant
- $P(\mathcal{F}(i) = HR)$ is a count on the utility matrix
- under the Naive assumption that all user ratings are independent

$$P(u | \mathcal{F}(i) = HR) \approx \prod_{r \in R} P(\mathcal{F}(i) = r | u)$$

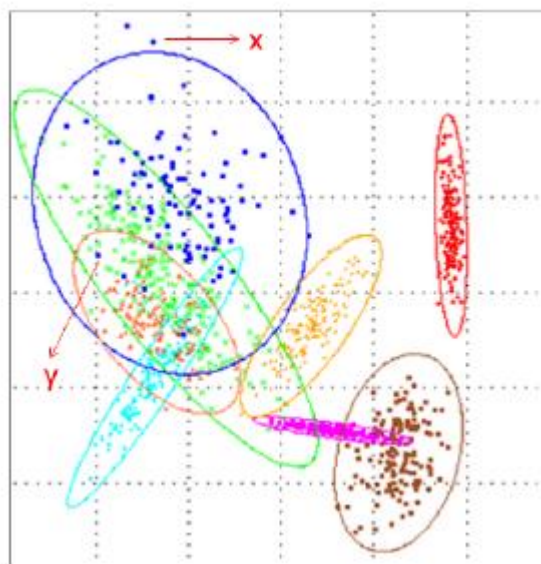
Machine Learning-based approaches: example of graph-based hybrid method

1. Represent user and content relationships in a graph
2. Enrich the model with semantic relationships (DBPedia <https://wiki.dbpedia.org/>)
3. Compute graph node embeddings using Random Walks on the graph
4. Estimate user/item similarities in the embedding space



Machine Learning-based approaches: example of clustering-based hybrid method

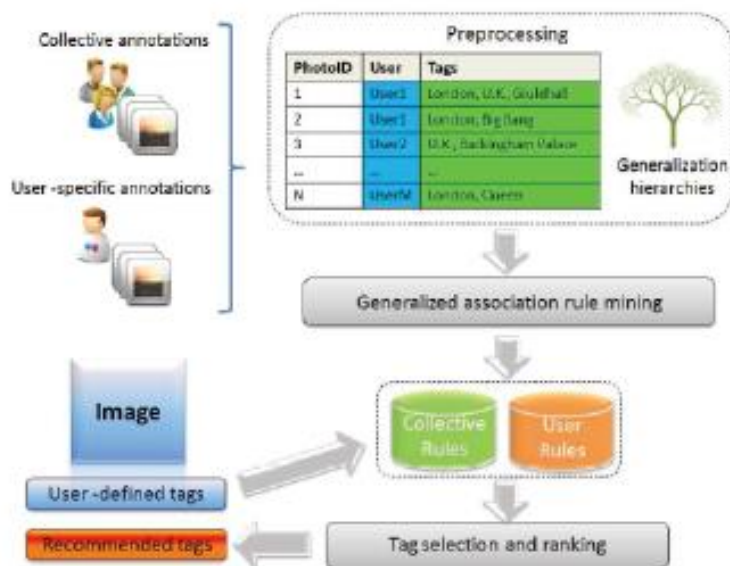
1. Apply knowledge expansion based on standard ontologies to generate item and user profiles
2. Cluster users and items separately using semantic similarity
3. Apply matrix factorization to predict the ratings



Mehrbakhsh Nilashi, Othman Ibrahim, Karamollah Bagherifard. A recommender system based on collaborative filtering using ontology and dimensionality reduction techniques, Expert Systems with Applications, 2018

Machine Learning-based approaches: example of association-based hybrid method

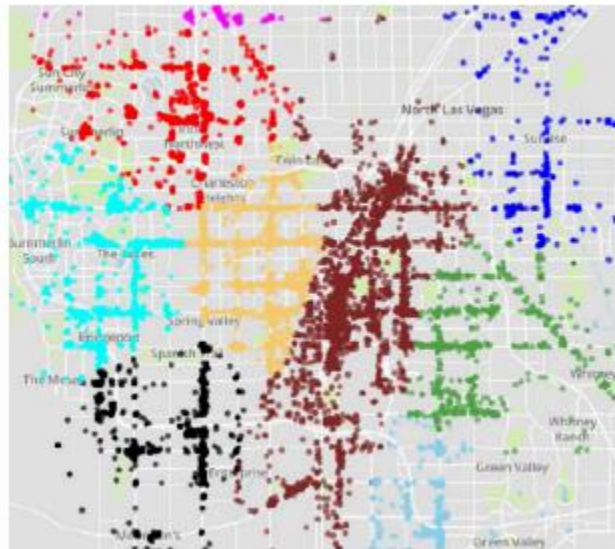
1. Recommend additional tags for partially annotated Flickr photos
2. Discover reliable associations among tags or tag categories from historical data
3. Recommend the most likely associated tags



Luca Cagliero, Alessandro Fiori, Luigi Grimaudo. 2014. Personalized tag recommendation based on generalized rules. ACM TIST 2014

Machine Learning-based approaches: example of RNN-based hybrid method

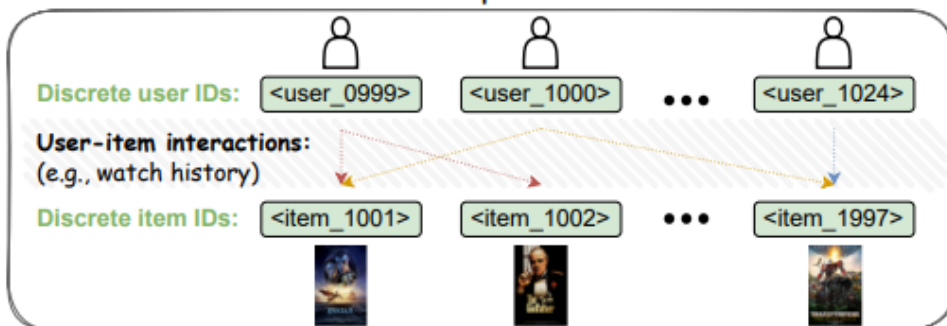
1. Recommend relevant Point-of-Interest considering the sequences of visited locations with assigned reviews and categories
2. Train a Recurrent Neural Network to predict the most likely next step
3. Divide the users into different groups and then train an RNN tailored to each group



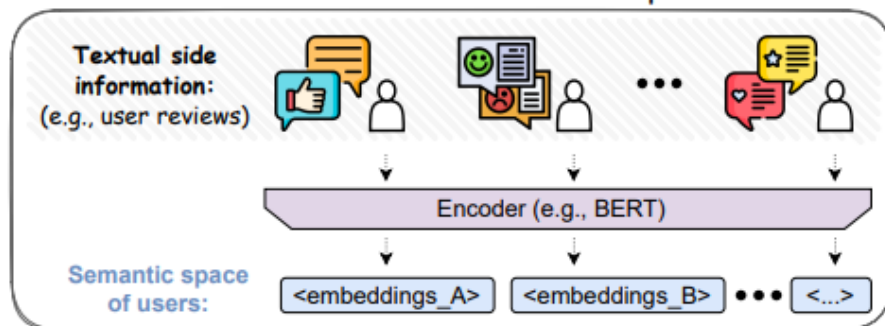
LLM-based approaches

- How to represent users and items

ID-based Representation

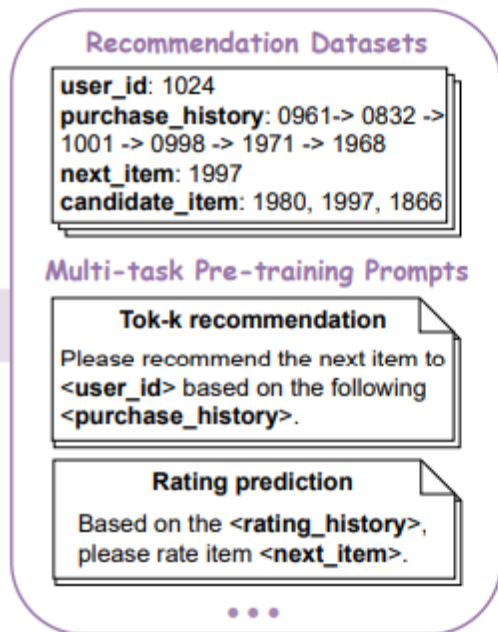


Textual Side Information-enhanced Representation



LLM-based approaches

- Workflow for pretraining a LLM for recommender systems using two different strategies
 - **Masked LM**: randomly mask tokens/spans and predict them based on the remaining context
 - **Next Token Prediction**: predict the next token based on the given context

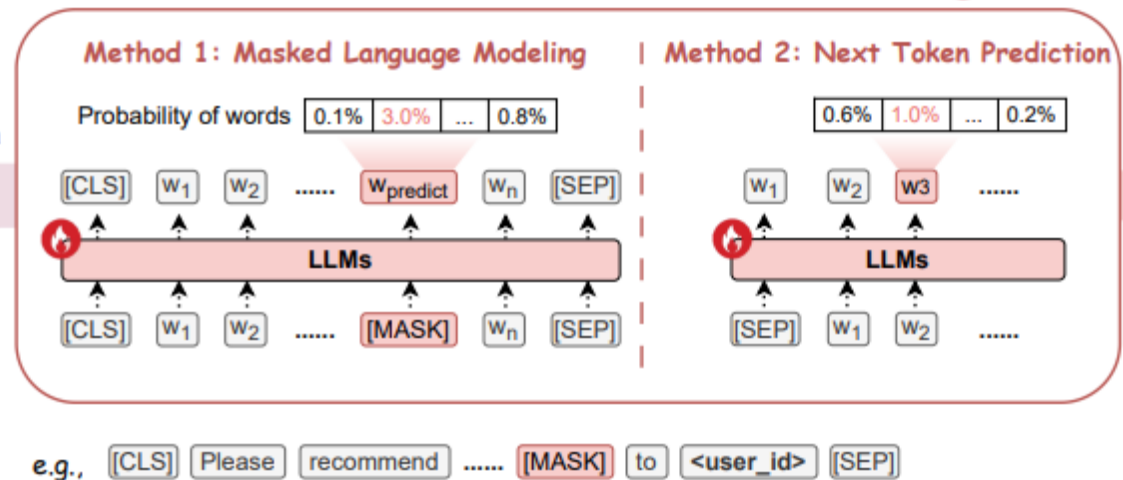


Large corpus
unlabeled data



Pre-training LLMs for Recommender Systems

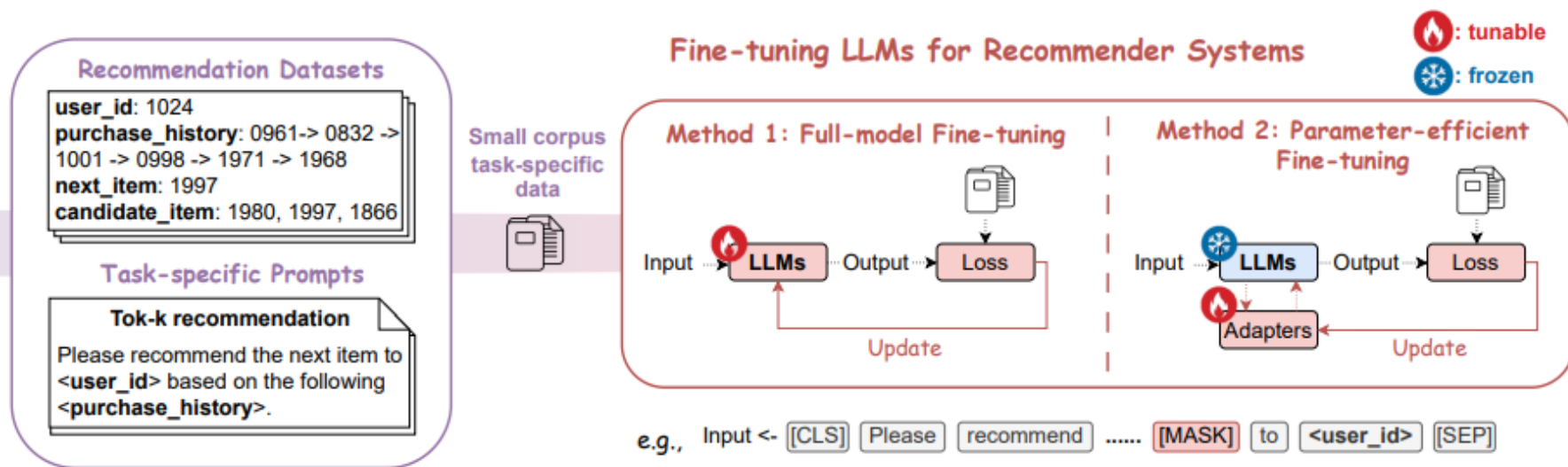
 : tunable



<https://arxiv.org/pdf/2307.02046>

LLM-based approaches

- Workflow for fine-tuning a LLM for recommender systems using full or lightweight methods

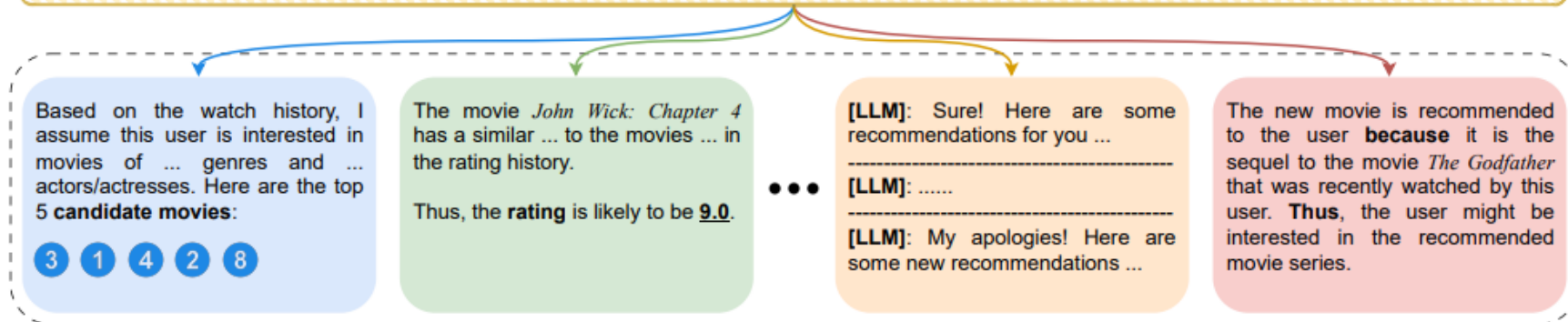
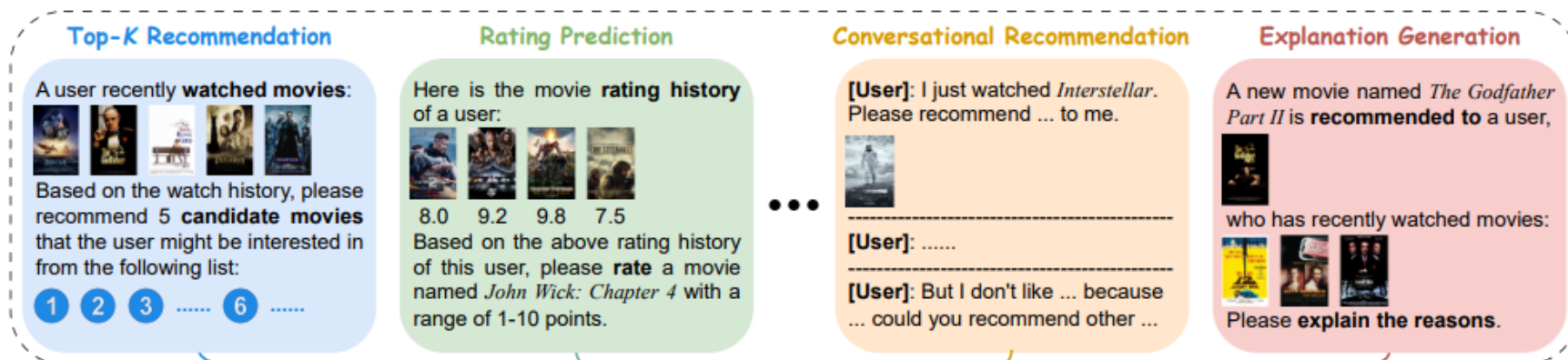


<https://arxiv.org/pdf/2307.02046>

LLM-based approaches

- Examples of movie recommendation scenarios

Task-specific Prompts (LLMs Inputs)



Task-specific Recommendations (LLMs Outputs)

<https://arxiv.org/pdf/2307.02046>

Results assessment

- Verify the quality of the predicted ratings
- Quantify errors, coverage, and diversity
- Compare alternative solutions (or the same solution with different configuration settings)

Benchmark datasets

- **MovieLens** (category: movies)
(<https://grouplens.org/datasets/movielens/>)
- **Million Song Dataset** (category: music)
(<http://millionsongdataset.com/>)
- **Amazon Review Data** (category: e-commerce)
(<https://nijianmo.github.io/amazon/index.html>)
- **Book-Crossing** (category: books)
(<http://www2.informatik.uni-freiburg.de/~cziegler/BX/>)

Benchmark dataset comparison

Dataset	Users	Items	Ratings	Density	Rating Scale
Movielens 1M	6040	3883	1,000,209	4.26%	[1-5]
Movielens 10M	69,878	10,681	10,000,054	1.33%	[0.5-5]
Movielens 20M	138,493	27,278	20,000,263	0.52%	[0.5-5]
Jester	124,113	150	5,865,235	31.50%	[-10, 10]
Book-Crossing	92,107	271,379	1,031,175	0.0041%	[1, 10], and implicit
Last.fm	1892	17632	92,834	0.28%	Play Counts
Wikipedia	5,583,724	4,936,761	417,996,366	0.0015%	Interactions
OpenStreetMap (Azerbaijan)	231	108,330	205,774	0.82%	Interactions
Gil (Django)	790	1757	13,165	0.95%	Interactions

Results assessment

- $F_p(u, i)$: predicted rating for user u and item i
- $F_a(u, i)$: actual rating for user u and item i
- Root Mean Square Error (RMSE)

$$RMSE = \sqrt{\sum_{u \in U, i \in I} (\mathcal{F}_p(u, i) - \mathcal{F}_a(u, i))^2}$$

Results assessment

- Precision at top K ($P@K$)
 - percentage of correct items in top K recommended items for a given user
- Rank correlation
 - Spearman's correlation between the vector of predicted ratings and those of actual ratings

Results assessment: binary prediction

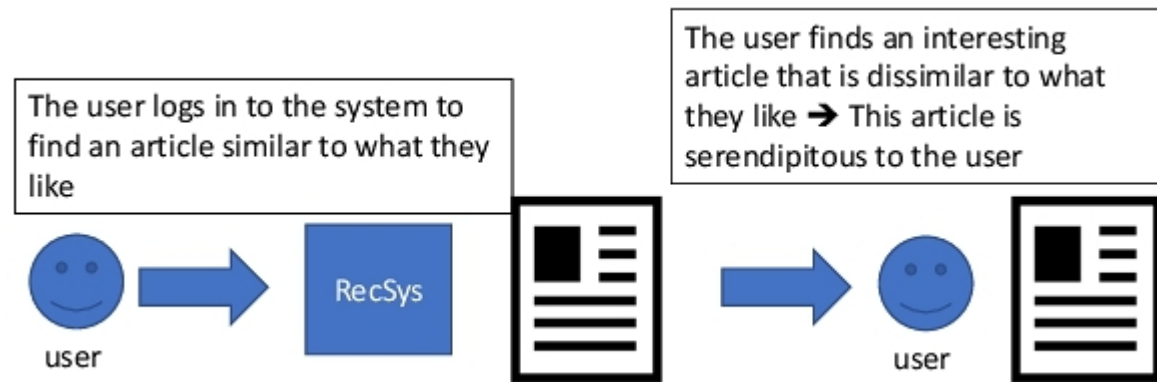
- Boolean rating: either relevant or not relevant
- Mean of Reciprocal Error $MRR = \max_{i \in I^r} \frac{1}{rank(i)}$
 - I^r is the set of relevant items
 - $rank(i)$ is the item rank in the recommended list
- Coverage
 - Number of items/users for which the recommender can make predictions
- Precision
 - Percentage of recommended relevant items
- Receiver operating characteristic (ROC)
 - Trade-off curve between false positives and false negatives

Results assessment: further aspects

- Diversity
 - diversify recommendations with the purposes of
 - Increase user satisfaction
 - Mitigate the potential effect of overfitting
- Domain adaptation
 - Tailor the recommender systems to the actual context
- Prediction order
 - refine the output rank according to domain-specific constraints
- Biased annotations
 - Ratings either do not reflect the actual service needs or miss a global viewpoint

Serendipity in recommender systems

- serendipity in a recommender is the experience of receiving an unexpected and fortuitous item recommendation.

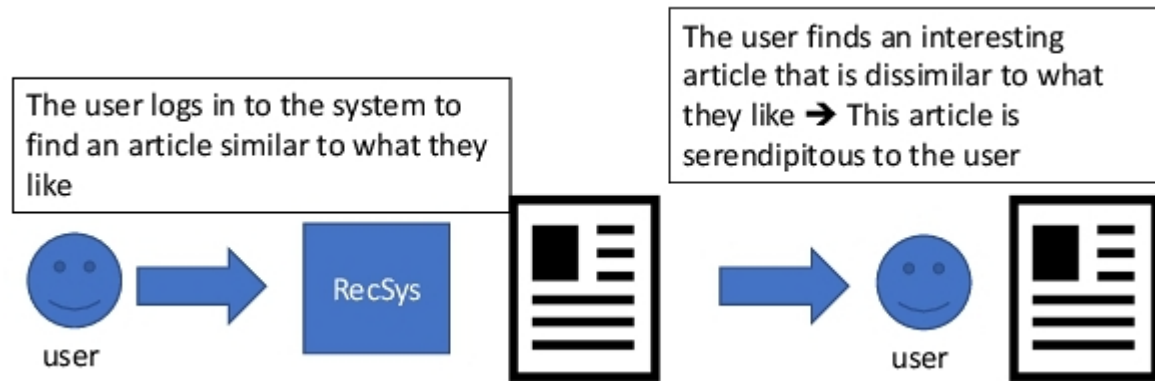


Serendipity in recommender systems

- The definition does not make any assumptions with respect to user goals, but assumes that users are looking for certain kinds of items, such as those they usually consume.
- Items can, however, be considered serendipitous regardless of what users are looking for and for what purpose, which can include browsing through items, influencing other users of the system or figuring out how the system works.
- Not knowing what the user's goals are and what the item will be used for will result in the misclassification of serendipitous and non-serendipitous items.

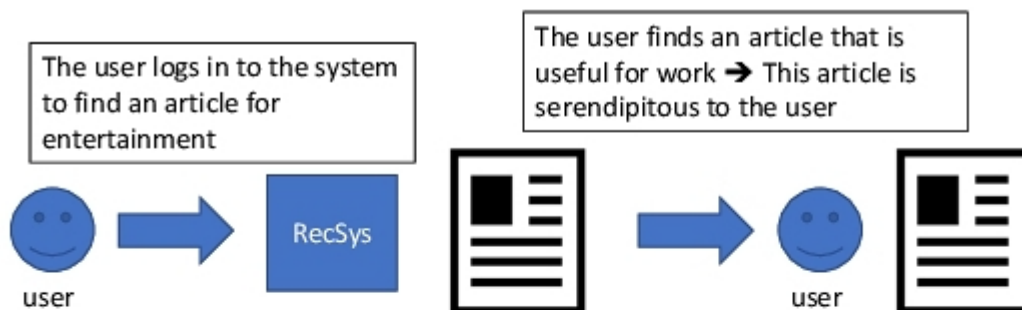
Serendipity in recommender systems

- the article is serendipitous to the user, only because it is relevant to the user and dissimilar to their tastes, while the user is looking for articles similar to their tastes.
- However, in practice, the user does not always look exclusively for articles similar to their tastes.

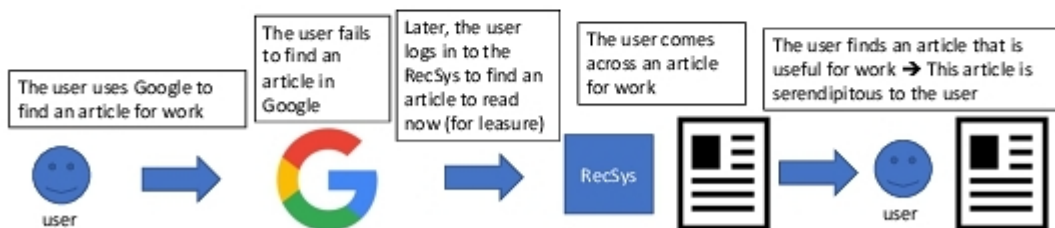


Serendipity in recommender systems

- An item is also considered serendipitous if it helps the user to achieve any goals that will appear in the future.



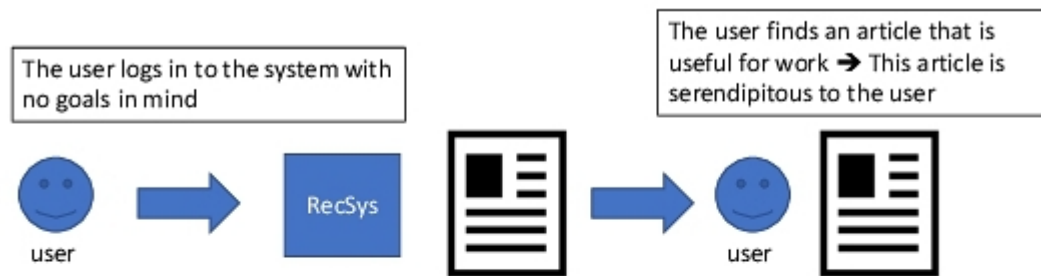
(a) Walpolian serendipity



(b) Mertonian serendipity

Serendipity in recommender systems

- An item is also considered serendipitous if it helps the user to achieve any goals that will appear in the future.

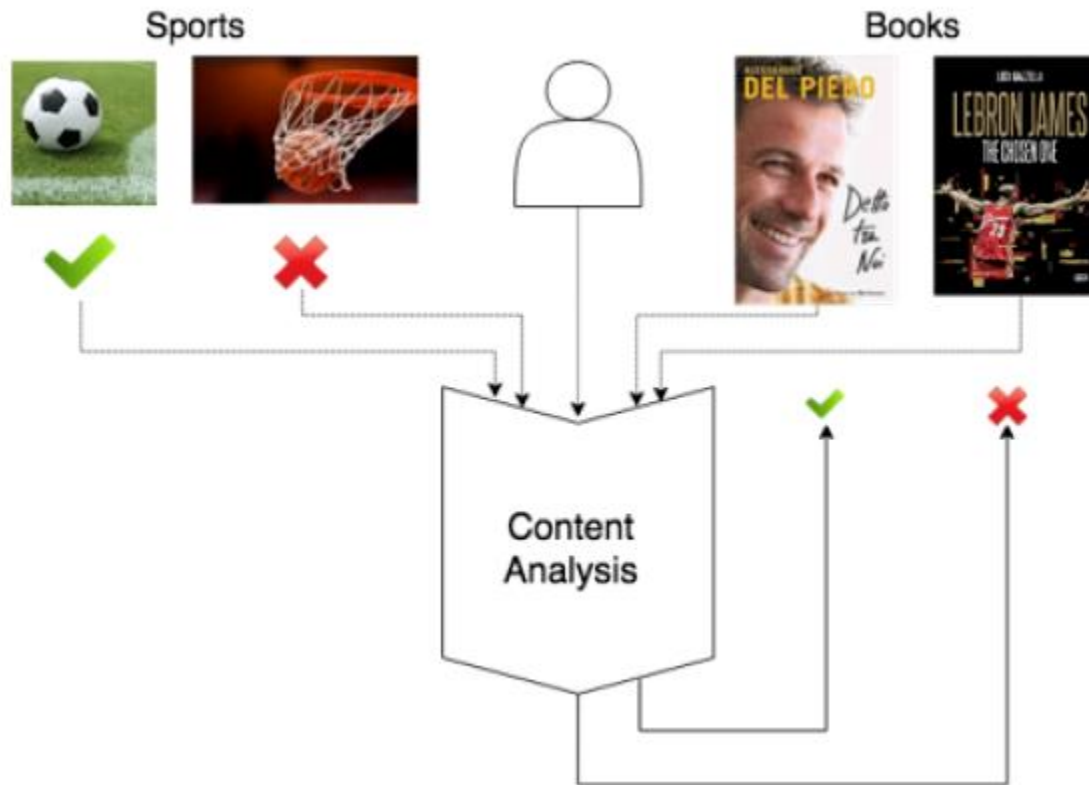


(c) Bushian serendipity



(d) Stephanian serendipity

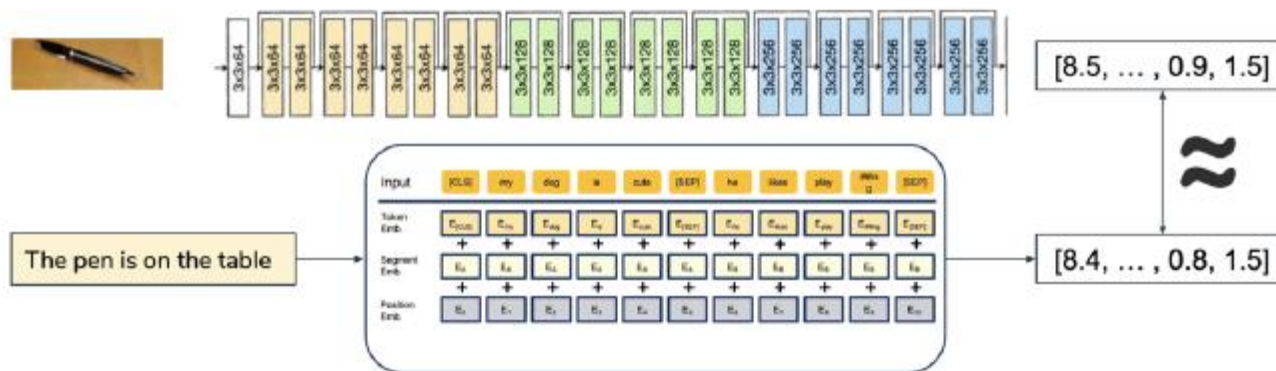
Challenges: contextualization



Challenges: transfer learning



Challenges: multimodal learning



Challenges: privacy and data protection



Challenges: real-time processing and scalability

- The greatest majority of the presented solutions are unsuitable for real time computation!
- Deep Learning solutions require dedicated hardware (GPUs)
- The inherent algorithm complexity limits the scalability towards Big Data (except for few cases)
- Take a look at <https://spark.apache.org/mllib/>

Acknowledgements and copyright license

- Copyright licence

- Attribution + Noncommercial + NoDerivatives



- Affiliation

- The author and his staff are currently members of the Database and Data Mining Group at Dipartimento di Automatica e Informatica (Politecnico di Torino) and of the SmartData interdepartmental centre
 - <https://dbdmg.polito.it>
 - <https://smartdata.polito.it>

Thank you!